

16-811: Math Fundamentals for Robotics, Fall 2025

Assignment 4

DUE: Thursday, October 30, 2025, by 11:59pm

1. Consider the following relationship between a function $y(x)$ and its derivative $y'(x)$ over the interval $[0, 1]$:

$$3y^2 y' = 1, \quad \text{with } y(1) = 1.$$

(Caution: The “initial condition” is specified at the right endpoint in this problem.)

- (a) By hand, obtain an exact analytic solution $y(x)$ to this differential equation.
(You may simply guess $y(x)$ and then show that it satisfies the differential equation.)

In the next three parts you will solve the differential equation numerically. You may **not** assume that you know the solution from part (a) within your solvers. Use part (a) merely to compute errors between your numerical solutions and the exact solution.

- (b) Implement and use Euler’s method to solve the differential equation numerically over the interval $[0, 1]$. Use a step size of 0.05. How accurate is your numerical solution?

(Compare the numerical solution to the exact solution, perhaps as follows: Create a table with one row for each x_i encountered using the given step size. The row might mention x_i , the true value $y(x_i)$, the estimated value y_i computed using Euler’s method, and the error $y(x_i) - y_i$. Graph the results as well. — You are about to implement some additional methods. You may wish to combine all the methods of this problem into one big table along with one composite graph.)

- (c) Implement and use a fourth-order Runge-Kutta method to solve the differential equation numerically. Again, use a step size of 0.05. Again, how accurate is your numerical solution?

- (d) Finally, implement and use fourth-order Adams-Bashforth for the differential equation. Again, use a step size of 0.05. Initialize the iteration with the following four values:

$$y(1.15) = 1.04768955317165,$$

$$y(1.10) = 1.03228011545637,$$

$$y(1.05) = 1.01639635681485,$$

$$y(1) = 1.$$

Once again, how accurate is your numerical solution?

Please submit code for parts (b)–(d). Include in your pdf any relevant code snippets that you would like the TAs to see, along with your table and graph, plus your work for (a).

2. Consider the function $f(x, y) = x^3 + y^3 - 2x^2 + 3y^2 - 8$ (for real x and y).

- (a) Find all critical points of f by sketching in the (x, y) -plane the iso-contours $\frac{\partial f}{\partial x} = 0$ and $\frac{\partial f}{\partial y} = 0$. Then classify the critical points into local minima, local maxima, and saddle points by considering either nearby gradient directions or the Hessian of f .
- (b) By hand, show how steepest descent would behave starting from the point $(x, y) = (1, -1)$. Use the version of steepest descent that moves to the nearest local minimum on the negative gradient line (solve that line optimization analytically). How many such steepest descent steps are needed to converge to an overall local minimum of f ?

Please explain how you got your results and show your work. No code is needed or expected.

3. Let Q be a real symmetric positive definite $n \times n$ matrix. Show that any two eigenvectors of Q corresponding to *distinct* eigenvalues of Q are Q -orthogonal. Show this directly from the definition of eigenvector, *without* assuming eigenvectors are orthogonal in the usual sense.

No code is needed or expected for this problem. Please put your work into the pdf.

4. (a) Show that in the purely quadratic form of the conjugate gradient method, $d_k^T Q d_k = -d_k^T Q g_k$. Consequently, show that to obtain x_{k+1} from x_k one does not need to use Q explicitly, assuming one already has available the vectors g_k , Qg_k , and d_k .

(See the notes for notation:

- “Purely quadratic” means conjugate gradient applied to functions of the form $f(x) = c + b^T x + \frac{1}{2} x^T Q x$, with Q a real symmetric positive definite matrix.
- g_k is the gradient of $f(x)$ evaluated at x_k , and d_k is the descent direction along which the method moves from x_k .)

- (b) Show that in the purely quadratic form Qg_k can be computed by taking a unit step from x_k in the direction of the negative gradient and then evaluating the gradient there. Specifically, if $y_k = x_k - g_k$ and $p_k = \nabla f(y_k)$, then $Qg_k = g_k - p_k$.
- (c) Combine the results of parts (a) and (b) to derive a conjugate gradient method for general functions f in the spirit of the algorithm presented in class, but which does not require knowledge of the Hessian of f or a line search. (In class we used line search to find minima of f along particular directions.) You may assume that the underlying ideas developed in (a) and (b) for purely quadratic functions carry over to general functions when you use the Conjugate Gradient Algorithm that appears on page 22 of the Optimization notes. Since that algorithm is based on the algorithm of page 17 in the notes, you will want to adapt the formulas for α_k and β_k that appear on page 17.

No code is needed or expected for this problem. Please answer all parts in the pdf.

5. Find the rectangle of a given perimeter that has the greatest area, by solving the Lagrange multiplier first-order necessary conditions. Verify the second-order sufficiency conditions.

No code is needed or expected for this problem. Please put your work into the pdf.

6. In this problem, you are going to use trajectory optimization to find obstacle-free paths for a robot. You will start with an obstacle cost world and a straight line path that blasts through the obstacles, as in Figures 1 and 2 on page 4. We are providing some starter code to make it easier – see the files `trajectory_optimization.m` and `trajectory_optimization.py`. For your code, you mainly just need to add the actual optimization code to one of these files.

- (a) Starting from the straight line path that goes across the obstacle field as in Figure 2, perform gradient descent to find a path with locally optimal cost. You should represent the path as a sequence of 2D points, view that sequence as a higher-dimensional point in some finite-dimensional space, and take gradient steps in that space. Rather than optimize along the entire gradient line as we did in lecture, simply scale the negative gradient by 0.1 and take a corresponding step.

Remember, this is a path for your robot, so make sure you don’t accidentally optimize its start and goal as well; those should remain fixed.

Plot the path after one iteration — notice that it is moving away from the obstacles. Pick convergence criteria and plot the path after convergence. What happened to it?

- (b) To avoid the issue from part (a), you need to tell the optimizer that this is a path instead of some independent 2D points.

One possibility is to consider an additional cost term, $\min(\frac{1}{2} \|\xi_i - \xi_{i-1}\|^2)$, that tells each 2D point ξ_i to be close to the previous 2D point ξ_{i-1} on the path. This additional term is sometimes called a *smoothness* cost.

Augment your negative gradient from part (a) with a new negative gradient, obtained for each ξ_i from the *single* term $\frac{1}{2} \|\xi_i - \xi_{i-1}\|^2$.

(The gradient of this new term with respect to ξ_i is $(\xi_i - \xi_{i-1})$. Include this term for ξ_i , but ignore this term when taking a gradient with respect to any other ξ_j , including ξ_{i-1} . This means we are really specifying a vector field for the optimizer to follow rather than an overall cost function. Another perspective is that each 2D point on the path is optimizing its own cost function, chosen in a convenient way.)

Weight the negative obstacle gradient from part (a) with 0.8 and the new negative gradient term with 4, add them together, then again scale that vector by 0.1 and take a corresponding step. (Weights like this are never set in stone; if you wish, you should feel free to experiment with other values.)

Plot the path after 100, 200 and 500 iterations. Why doesn't this work?

- (c) Finally, try augmenting the obstacle gradient with a different smoothness cost, telling each 2D point to be close to both its previous and next points on the path:

$$\min \left(\frac{1}{2} \|\xi_i - \xi_{i-1}\|^2 + \frac{1}{2} \|\xi_{i+1} - \xi_i\|^2 \right).$$

Taking the gradient with respect to ξ_i yields $(\xi_i - \xi_{i-1}) + (\xi_i - \xi_{i+1})$, which is $-\xi_{i-1} + 2\xi_i - \xi_{i+1}$. (We can now view this gradient as derived from an overall cost function or we can again view it as derived for each 2D point from its own cost function.) Perform another optimization using this new smoothness cost to compute gradient updates:

- i. Use 0.8 to weight the obstacle cost and 4 to weight the smoothness cost. Then use a step-size scaling of 0.1 as before.
- ii. Again, you should feel free to use different weights if your code works better with those, so long as the weights are not too different from those we suggested.

Plot the path after 100 and 5000 iterations. Why is this version better?

- (d) What will happen if you were to start the procedure from different initial paths? Will the final answer be the same every time? Why?
- (e) Suppose that the final solution path found by gradient descent, given some initial path and given our original cost function, is not feasible for the robot to execute (it still passes through some high cost regions that you would rather not have the robot pass through)? What simple common-sense strategy can you come up with to try to mitigate this problem? (*No need to write code unless necessary. We expect a very short description of the strategy only.*)

Please submit code for parts (a), (b), and (c), but none is required for (d) and (e). Please provide answers for all parts in your pdf, as usual. In particular, replicate any code snippets in your pdf that you think the TAs will need for grading.

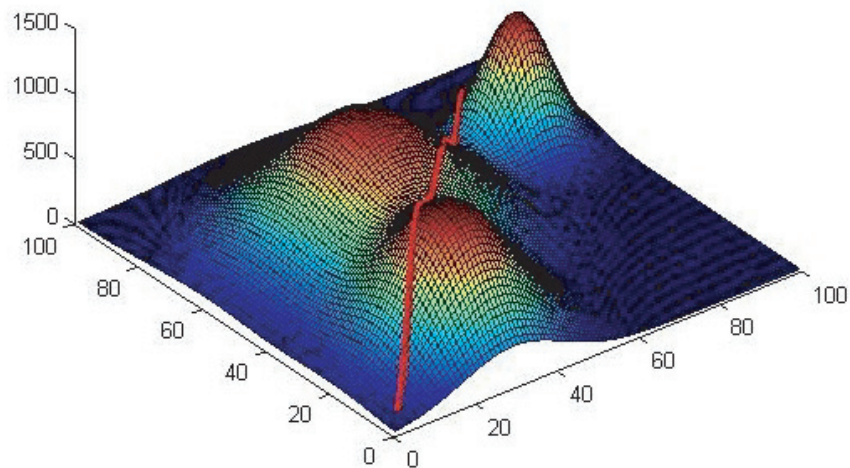


Figure 1: A 2D world with obstacles depicted as cost functions. Shown in red is a straight line path between two locations. The path has high cost because it passes through/near obstacles.

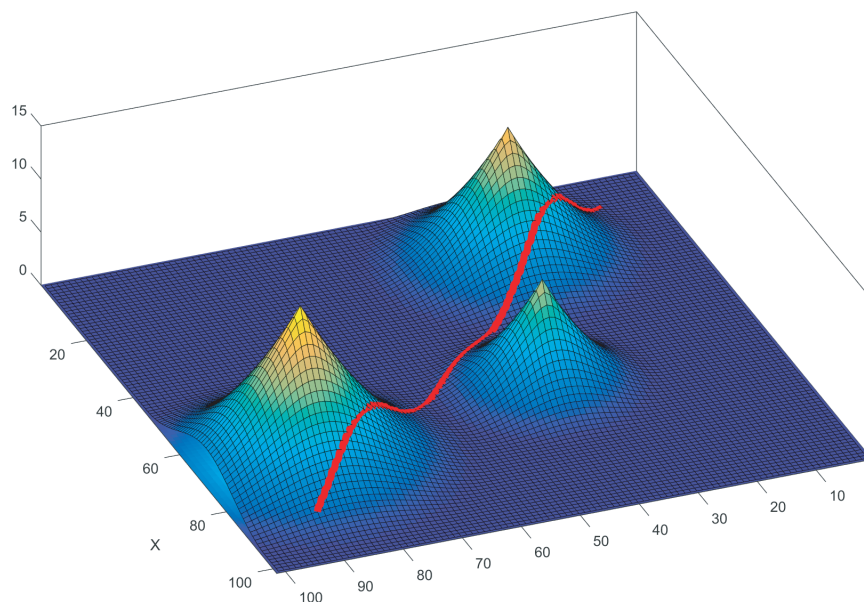


Figure 2: Another 2D obstacle world and straight line path with moderately-high cost, corresponding to the code appearing in `trajectory_optimization.mpy`.

1. $3y^2 y' = 1$ $y(1) = 1$ interval $[0, 1]$.

$$3y^2 \frac{dy}{dx} = 1$$

$$3y^2 dy = dx$$

$$\int 3y^2 dy = \int dx$$

$$y^3 = x + C.$$

$$y(1) = 1 \quad 1 = 1 + C \quad C = 0$$

$$\Rightarrow y^3 = x \quad y = \sqrt[3]{x}.$$

HW4 Question 1

```
clear; clc;

f = @(x,y) 1./(3*(y.^2));
y_exact = @(x) x.^(1/3);           % Part a

h = -0.05;                         % backward step (1 -> 0) with |h|=0.05
x_grid = 1:-0.05:0;               % 1.00, 0.95, ..., 0.00 (inclusive)
N = numel(x_grid);
```

part b

```
y_euler = nan(N,1);
y_euler(1) = 1.0;                  % y(1)=1
for i = 1:N-1
    yi = y_euler(i);
    % guard against division by zero / negative crossing near x=0
    if yi <= 0, yi = max(yi, 1e-12); end
    y_euler(i+1) = yi + h*f(x_grid(i), yi);
end
```

part c

```
y_rk4 = nan(N,1);
y_rk4(1) = 1.0;
for i = 1:N-1
    xi = x_grid(i); yi = y_rk4(i);
    yi = max(yi, 1e-12);
    k1 = f(xi, yi);
    k2 = f(xi + h/2, yi + (h/2)*k1);
    k3 = f(xi + h/2, yi + (h/2)*k2);
    k4 = f(xi + h, yi + h*k3);
    y_rk4(i+1) = yi + (h/6)*(k1 + 2*k2 + 2*k3 + k4);
end
```

part d

```
x_ab4 = 1.15:-0.05:0;              % starts at 1.15 to use seeds then march
to 0
M = numel(x_ab4);
y_ab4 = nan(M,1);

% Given seeds (align to first four entries: 1.15, 1.10, 1.05, 1.00)
seed_x = [1.15 1.10 1.05 1.00];
seed_y = [1.04768955317165, 1.03228011545637, 1.01639635681485, 1.00000000000000];
y_ab4(1:4) = seed_y(:);
```

```

% AB4 recurrence:  $y_{n+1} = y_n + h/24 * (55 f_n - 59 f_{n-1} + 37 f_{n-2} - 9 f_{n-3})$ 
fn = f(x_ab4(n), max(y_ab4(n), 1e-12));
fn_1 = f(x_ab4(n-1), max(y_ab4(n-1), 1e-12));
fn_2 = f(x_ab4(n-2), max(y_ab4(n-2), 1e-12));
fn_3 = f(x_ab4(n-3), max(y_ab4(n-3), 1e-12));
y_ab4(n+1) = y_ab4(n) + (h/24)*(55*fn - 59*fn_1 + 37*fn_2 - 9*fn_3);
end

% Extract AB4 values on the same x_grid (1.00, 0.95, ..., 0.00)
% Note: x_ab4 sequence includes 1.15 seed and then 1.10, 1.05, 1.00, 0.95, ..., 0.00
[~, ia, ib] = intersect(round(x_ab4,10), round(x_grid,10), 'stable');
y_ab4_on_grid = nan(N,1);
y_ab4_on_grid(ib) = y_ab4(ia);

%-----
% Exact values and errors
%-----
y_true = y_exact(x_grid).';
err_euler = y_true - y_euler;
err_rk4 = y_true - y_rk4;
err_ab4 = y_true - y_ab4_on_grid;

%-----
% Assemble table
%-----
T = table( ...
    x_grid.', y_true, y_euler, err_euler, y_rk4, err_rk4, y_ab4_on_grid, err_ab4,
    ...
    'VariableNames',
    {'x', 'y_exact', 'y_euler', 'err_euler', 'y_rk4', 'err_rk4', 'y_ab4', 'err_ab4'} ...
);

disp('==== Q1: Exact vs Euler / RK4 / AB4 (h = 0.05 backward) ====');

```

```
==== Q1: Exact vs Euler / RK4 / AB4 (h = 0.05 backward) ====
```

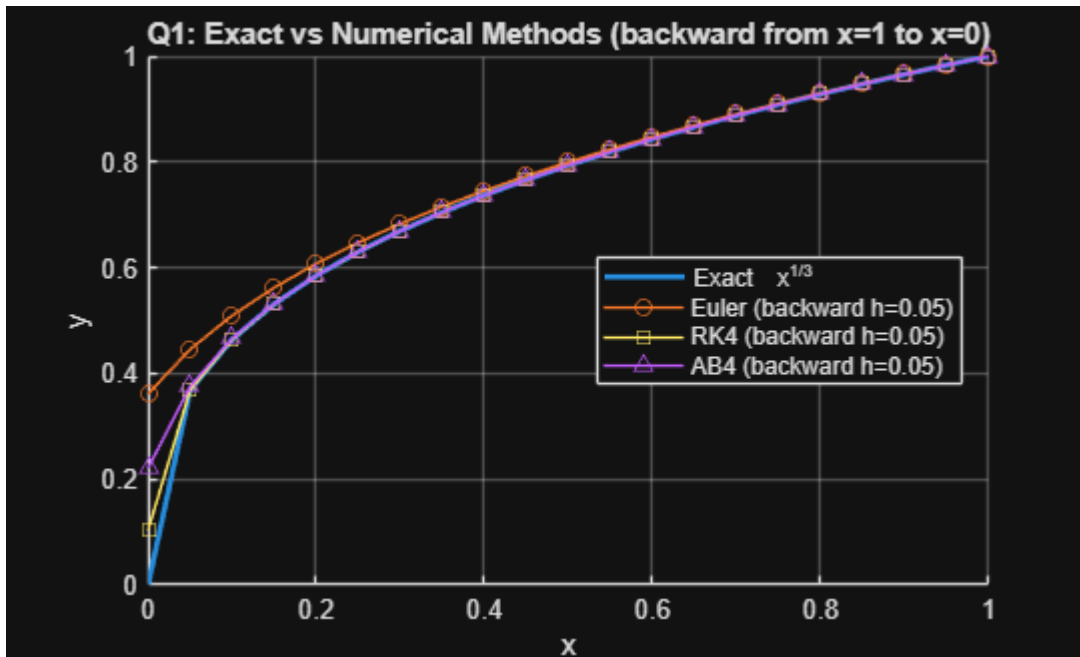
```
disp(T);
```

x	y_exact	y_euler	err_euler	y_rk4	err_rk4	y_ab4	err_ab4
1	1	1	0	1	0	1	0
0.95	0.98305	0.98333	-0.00028576	0.98305	2.3155e-10	0.98305	-3.1327e-07
0.9	0.96549	0.9661	-0.00060752	0.96549	5.3542e-10	0.96549	-7.3196e-07
0.85	0.94727	0.94824	-0.00097172	0.94727	9.3811e-10	0.94727	-1.2672e-06
0.8	0.92832	0.9297	-0.0013863	0.92832	1.478e-09	0.92832	-1.9754e-06
0.75	0.90856	0.91042	-0.0018615	0.90856	2.2119e-09	0.90856	-2.9209e-06
0.7	0.8879	0.89031	-0.00241	0.8879	3.226e-09	0.88791	-4.2001e-06
0.65	0.86624	0.86929	-0.0030487	0.86624	4.6545e-09	0.86625	-5.9595e-06
0.6	0.84343	0.84723	-0.0037994	0.84343	6.7129e-09	0.84344	-8.4267e-06
0.55	0.81932	0.82401	-0.0046917	0.81932	9.7605e-09	0.81933	-1.1966e-05

0.5	0.7937	0.79947	-0.0057665	0.7937	1.4421e-08	0.79372	-1.7181e-05
0.45	0.76631	0.77339	-0.0070812	0.76631	2.1834e-08	0.76633	-2.5119e-05
0.4	0.73681	0.74553	-0.0087198	0.73681	3.421e-08	0.73684	-3.7679e-05
0.35	0.70473	0.71554	-0.01081	0.70473	5.616e-08	0.70479	-5.8535e-05
0.3	0.66943	0.68299	-0.013555	0.66943	9.8258e-08	0.66953	-9.5319e-05
0.25	0.62996	0.64726	-0.017298	0.62996	1.8783e-07	0.63013	-0.00016547
0.2	0.5848	0.60748	-0.022672	0.5848	4.0812e-07	0.58512	-0.00031407
0.15	0.53133	0.56231	-0.030983	0.53133	1.0799e-06	0.53201	-0.00067928
0.1	0.46416	0.5096	-0.045443	0.46415	3.9945e-06	0.46597	-0.0018083
0.05	0.3684	0.44542	-0.077021	0.36837	2.8738e-05	0.37551	-0.007106
0	0	0.36142	-0.36142	0.10346	-0.10346	0.22082	-0.22082

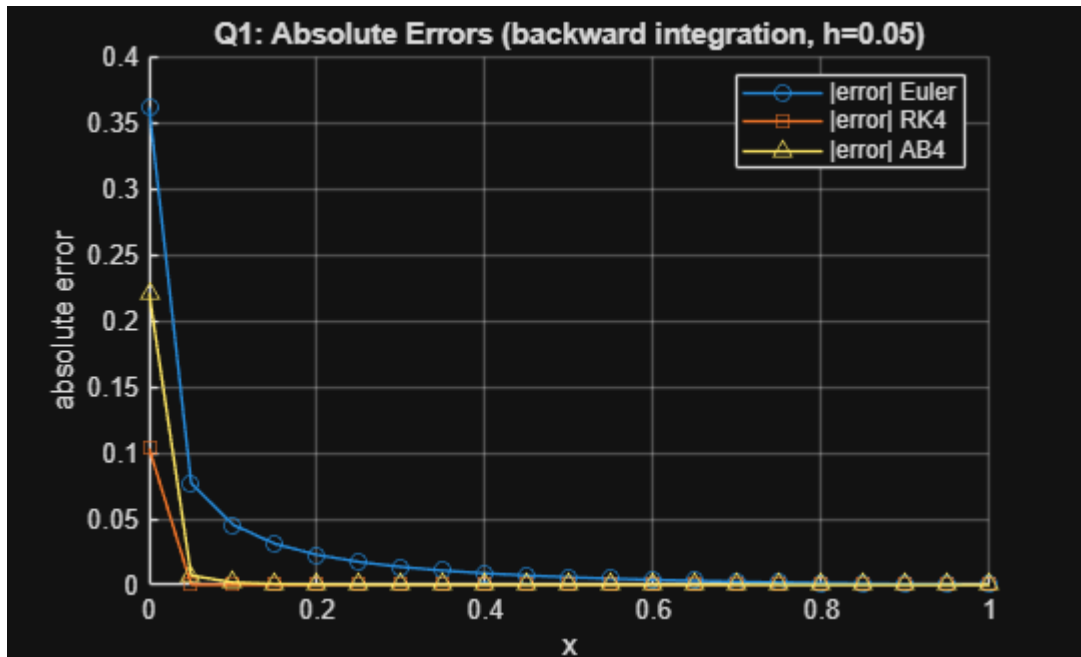
Plot

```
figure(1); clf; hold on; grid on;
plot(x_grid, y_true, 'LineWidth', 2, 'DisplayName','Exact x^{1/3}');
plot(x_grid, y_euler, 'o-', 'DisplayName','Euler (backward h=0.05)');
plot(x_grid, y_rk4, 's-', 'DisplayName','RK4 (backward h=0.05)');
plot(x_grid, y_ab4_on_grid, '^-', 'DisplayName','AB4 (backward h=0.05)');
xlabel('x'); ylabel('y');
title('Q1: Exact vs Numerical Methods (backward from x=1 to x=0)');
legend('Location','best');
```



```
%-----
% Plot absolute errors (semilogy to handle wide range)
%-----
figure(2); clf; hold on; grid on;
semilogy(x_grid, abs(err_euler), 'o-', 'DisplayName','|error| Euler');
semilogy(x_grid, abs(err_rk4), 's-', 'DisplayName','|error| RK4');
semilogy(x_grid, abs(err_ab4), '^-', 'DisplayName','|error| AB4');
xlabel('x'); ylabel('absolute error');
```

```
title('Q1: Absolute Errors (backward integration, h=0.05)');
legend('Location','best');
```



```
%-----
% Brief accuracy summary in Command Window
%-----
fprintf('\nMax |error| (Euler): %g\n', max(abs(err_euler),[],'omitnan'));
```

```
Max |error| (Euler): 0.361419
```

```
fprintf('Max |error| (RK4) : %g\n', max(abs(err_rk4),[],'omitnan'));
```

```
Max |error| (RK4) : 0.103455
```

```
fprintf('Max |error| (AB4) : %g\n\n', max(abs(err_ab4),[],'omitnan'));
```

```
Max |error| (AB4) : 0.220816
```

$$2. \quad f(x, y) = x^3 + y^3 - 2x^2 + 3y^2 - 8 \quad (\text{red } x \text{ \& } y).$$

$$\frac{\partial f}{\partial x} = 3x^2 - 4x = x(3x - 4) \quad \frac{\partial f}{\partial y} = 3y^2 + 6y = y(3y + 6)$$

$$\text{zero: } x \in \{0, \frac{4}{3}\} \\ y \in \{0, -2\}.$$

$$\text{critical point: } (0, 0), (0, -2), (\frac{4}{3}, 0), (\frac{4}{3}, -2).$$

$$H = \begin{bmatrix} f_{xx} & f_{xy} \\ f_{yx} & f_{yy} \end{bmatrix} = \begin{bmatrix} 6x - 4 & 0 \\ 0 & 6y + 6 \end{bmatrix} \quad D(H) = (6x - 4)(6y + 6).$$

$$(0, 0): f_{xx} = -4 \quad f_{yy} = 6 \quad D = -24 < 0 \quad \text{saddle}$$

$$(0, -2): f_{xx} = -4 \quad f_{yy} = -6 \quad D = 24 > 0 \quad f_{xx} < 0 \quad \text{local max}$$

$$(\frac{4}{3}, 0): f_{xx} = 4 \quad f_{yy} = 6 \quad D = 24 > 0 \quad f_{xx} > 0 \quad \text{local min.}$$

$$(\frac{4}{3}, -2): f_{xx} = 4 \quad f_{yy} = -6 \quad D = -24 < 0 \quad \text{saddle}$$

$$b). \quad \nabla f(x, y) = \begin{pmatrix} 3x^2 - 4x \\ 3y^2 + 6y \end{pmatrix}.$$

$$\text{for } (x, y) = (1, -1) \quad \nabla f(x, y) = \begin{pmatrix} -1 \\ -3 \end{pmatrix} = u.$$

$$\nabla f(1, -1) = [-1, -3]. \quad -\nabla f = [1, 3].$$

$$(x, y) = (1 + \alpha, -1 + 3\alpha)$$

$$f(1 + \alpha, -1 + 3\alpha) = (1 + \alpha)^3 + (-1 + 3\alpha)^3 - 2(1 + \alpha)^2 + 3(-1 + 3\alpha)^2 - 8 \\ = 28\alpha^3 + \alpha^2 - 10\alpha - 7.$$

$$\frac{\partial f}{\partial \alpha} = 84\alpha^2 + 2\alpha - 10 = 0 \quad \Rightarrow \quad \alpha \in \left\{ \frac{1}{3}, -\frac{5}{4} \right\}$$

$$\frac{\partial^2 f}{\partial \alpha^2} = 168\alpha + 2 \quad \text{for } \alpha = \frac{1}{3} \quad \frac{\partial^2 f}{\partial \alpha^2} = 58 > 0 \quad \Rightarrow \text{min}$$

$$\text{for } \alpha = -\frac{5}{4} \quad \frac{\partial^2 f}{\partial \alpha^2} = -58 < 0 \quad \Rightarrow \text{max}$$

$$(x, y) = (1 + \frac{1}{3}, -1 + 3 \times \frac{1}{3}) = (\frac{4}{3}, 0) \Rightarrow \text{local min}$$

$$\text{When } (\frac{4}{3}, 0), \quad \nabla f = (3x^2 - 4x, 3y^2 + 6y) = (0, 0)$$

Behavior: From $(1, -1)$ with exact line search along $(1, 3)$

to $(\frac{4}{3}, 0)$ with step $\alpha = \frac{1}{3}$ by min=1 step.

3. Let Q be a real symmetric positive definite $n \times n$ matrix. Show that any two eigenvectors of Q corresponding to *distinct* eigenvalues of Q are Q -orthogonal. Show this directly from the definition of eigenvector, *without* assuming eigenvectors are orthogonal in the usual sense.

No code is needed or expected for this problem. Please put your work into the pdf.

$$Q \in \mathbb{R}^{n \times n}. \quad Q v_i = \lambda_i v_i, \quad \lambda_i \neq \lambda_j.$$

$$Q v_j = \lambda_j v_j.$$

$$v_j^T Q v_i = (Q v_i)^T v_j = (\lambda_i v_i)^T v_j = \lambda_i v_i^T v_j.$$

$$v_i^T Q v_j = v_i^T (Q v_j) = v_i^T (\lambda_j v_j) = \lambda_j v_i^T v_j.$$

$$(\lambda_i - \lambda_j) v_i^T v_j = 0 \quad \text{since } \lambda_i \neq \lambda_j \Rightarrow v_i^T v_j = 0.$$

$$v_i^T Q v_j = \lambda_j v_i^T v_j = 0.$$

Thus any two eigenvectors of Q corresponding to distinct eigenvalues of Q are Q -orthogonal.

4. (a) Show that in the purely quadratic form of the conjugate gradient method, $d_k^T Q d_k = -d_k^T Q g_k$. Consequently, show that to obtain x_{k+1} from x_k one does not need to use Q explicitly, assuming one already has available the vectors g_k , Qg_k , and d_k .

(See the notes for notation:

- "Purely quadratic" means conjugate gradient applied to functions of the form $f(x) = c + b^T x + \frac{1}{2} x^T Q x$, with Q a real symmetric positive definite matrix.
- g_k is the gradient of $f(x)$ evaluated at x_k , and d_k is the descent direction along which the method moves from x_k .)

- (b) Show that in the purely quadratic form Qg_k can be computed by taking a unit step from x_k in the direction of the negative gradient and then evaluating the gradient there. Specifically, if $y_k = x_k - g_k$ and $p_k = \nabla f(y_k)$, then $Qg_k = g_k - p_k$.

- (c) Combine the results of parts (a) and (b) to derive a conjugate gradient method for general functions f in the spirit of the algorithm presented in class, but which does not require knowledge of the Hessian of f or a line search. (In class we used line search to find minima of f along particular directions.) You may assume that the underlying ideas developed in (a) and (b) for purely quadratic functions carry over to general functions when you use the Conjugate Gradient Algorithm that appears on page 22 of the Optimization notes. Since that algorithm is based on the algorithm of page 17 in the notes, you will want to adapt the formulas for α_k and β_k that appear on page 17.

No code is needed or expected for this problem. Please answer all parts in the pdf.

$$a), d_k^T Q d_k = -d_k^T Q g_k.$$

$$f(x) = c + b^T x + \frac{1}{2} x^T Q x \quad \text{with } Q = Q^T \succeq 0.$$

$$\text{Then } g_k = \nabla f(x_k) = b + Q x_k.$$

1. Orthogonality of consecutive gradients:

$$g_{k+1}^T g_k = 0.$$

$$\text{Since } g_{k+1} = g_k + \alpha_k Q d_k.$$

$$\Rightarrow 0 = (g_k + \alpha_k Q d_k)^T g_k = \|g_k\|^2 + \alpha_k d_k^T Q g_k$$

$$\Rightarrow \alpha_k = - \frac{\|g_k\|^2}{d_k^T Q g_k}.$$

2. Orthogonality of g_k

$$g_k^T d_{k+1} = 0 \Rightarrow \text{from } d_{k+1} = -g_k + \beta_k d_k.$$

$$g_k^T d_k = -\|g_k\|^2$$

$$0 = (g_k + \alpha_k Q d_k)^T d_k = g_k^T d_k + \alpha_k d_k^T Q d_k$$

$$\Rightarrow \alpha_k = - \frac{g_k^T d_k}{d_k^T Q g_k}$$

$$\text{for } \alpha_k = - \frac{\|g_k\|^2}{d_k^T Q g_k} = \frac{\|g_k\|^2}{d_k^T Q g_k}$$

$$\therefore -d_k^T Q g_k = d_k^T Q g_k.$$

$$\alpha = \frac{\|g_k\|^2}{-d_k^T Q g_k}.$$

b). For quadratic f .

$$y_k = x_k - g_k \quad p_k = \nabla f(y_k).$$

$$\nabla f(x) = b + Qx.$$

$$p_k = b + Q(x_k - g_k)$$

$$= (b + Qx_k) - Qg_k$$

$$= g_k - Qg_k.$$

$$\text{thus } \Rightarrow Qg_k = g_k - p_k = g_k - \nabla f(x_k - g_k)$$

thus Qg_k is obtained by a unit step along $-g_k$
and one more gradient evaluation.
no explicit Q .

c).

$$x_{k+1} = x_k + \frac{d_k^T g_k}{d_k^T Q g_k} d_k$$

$$= x_k + \frac{d_k^T g_k}{d_k^T (g_k - p_k)} d_k$$

$$= x_k + \frac{d_k^T g_k}{d_k^T g_k - d_k^T p_k} d_k.$$

Since we know

$$d_k^T g_k = d_k^T (Q(x_0 + \alpha_0 d_0 + \dots + \alpha_{k-1} d_{k-1}) + b)$$

$$= d_k^T Qx_0 + \alpha_0 d_k^T Qd_0 + \dots + \alpha_{k-1} d_k^T Qd_{k-1} + d_k^T b.$$

$$= d_k^T g_0.$$

$$\therefore x_{k+1} = x_k + \frac{d_k^T g_0}{d_k^T g_0 - d_k^T p_k} d_k \quad \text{does not require}$$

knowledge of the Hessian of f
or a line search.

5. Find the rectangle of a given perimeter that has the greatest area, by solving the Lagrange multiplier first-order necessary conditions. Verify the second-order sufficiency conditions.

No code is needed or expected for this problem. Please put your work into the pdf.

Assume perimeter $= c > 0$ side length be $x, y > 0$.

$$\max f(x, y) = xy \Rightarrow h(x, y) = 2x + 2y - c = 0.$$

$$\mathcal{L}(x, y, \lambda) = xy - \lambda(2x + 2y - c).$$

$$\partial_x \mathcal{L} = y - 2\lambda = 0, \quad \partial_y \mathcal{L} = x - 2\lambda = 0, \quad 2x + 2y - c = 0.$$

$$2(2\lambda) + 2(2\lambda) - c = 0 \quad \lambda = \frac{c}{8} \Rightarrow xy = \frac{c^2}{16}$$

$$\text{Area}_{\max} = xy = \frac{c^2}{16}$$

$$\nabla_{xy}^2 \mathcal{L} = \begin{pmatrix} 0 & 1 \\ 1 & 0 \end{pmatrix} \quad \nabla h = (2, 2)^T.$$

$$\mathcal{T} = \{v \in \mathbb{R}^2 : \nabla h^T v = 0\} = \{c u, -u\} : u \in \mathbb{R}\}$$

for any nonzero $v = (u, -u) \in \mathcal{T}$.

$$v^T \nabla_{xy}^2 \mathcal{L} v = (u, -u) \begin{pmatrix} 0 & 1 \\ 1 & 0 \end{pmatrix} \begin{pmatrix} u \\ -u \end{pmatrix} = -2u^2 < 0$$

Hence the Hessian of $\mathcal{L}$ is restricted to the tangent space is negative definite. verifying the second-order sufficiency for a strict constrained maximum.

Conclusion: The rectangle of greatest area with fixed perimeter c is a square with side $c/4$ and area $c^2/16$.

Question 6

part a

```
%% create obstacle field
close all
clear all
waypoints=300;
N=101;
OBST = [20,30;60,40;70,85];
epsilon = [25; 20; 30];

obs_cost = double(zeros(N));
for i=1:size(OBST,1)

    t = zeros(N);
    t(OBST(i,1),OBST(i,2)) = 1; %point obstacles

    t_cost = double(bwdist(t));
    t_cost(t_cost>epsilon(i))=epsilon(i);
    t_cost = 1/(2*epsilon(i))*(t_cost-epsilon(i)).^2;

    obs_cost = obs_cost + t_cost(1:N, 1:N);
end

figure(1)
imagesc(obs_cost')
% obstacle cost gradient
gx = diff(double(obs_cost),1,1);
gy = diff(double(obs_cost),1,2);
hold on

figure(1);
surface(1:N,1:N,double(obs_cost'));
xlabel('X')
hold on;

%% initial path
%world params
SX = 10; % START
SY = 10;
GX = 90; % GOAL
GY = 90;

traj = zeros(2,waypoints);
traj(1,1)=SX;
traj(2,1)=SY;
dist_x = GX-SX;
```

```

dist_y = GY-SY;
for i=2:waypoints
    traj(1,i)=traj(1,i-1)+dist_x/(waypoints-1);
    traj(2,i)=traj(2,i-1)+dist_y/(waypoints-1);
end

path_init = traj';
tt=size(path_init,1);
path_init_values = zeros(size(path_init,1),1);
for i=1:tt
    path_init_values(i)=obs_cost(floor(path_init(i,1)),floor(path_init(i,2)));
end
plot3(path_init(:,1),path_init(:,2),path_init_values,'.r','MarkerSize',20);
hold on

path = path_init;

%% Optimize it...
% your code comes here

%% Optimize it... (Part a: obstacle cost only)
scale = 0.1;
max_iter = 200;

for iter = 1:max_iter
    for i = 2:tt-1
        x = path(i,1); y = path(i,2);
        % skip if out of grid bounds
        if x < 1 || x > N-1 || y < 1 || y > N-1
            continue;
        end
        % bilinear interpolation for gx, gy
        interp_x1 = (x - fix(x))*(gx(fix(x)+1, fix(y)) - gx(fix(x), fix(y))) +
gx(fix(x), fix(y));
        interp_x2 = (x - fix(x))*(gx(fix(x)+1, fix(y)+1) - gx(fix(x), fix(y)+1)) +
gx(fix(x), fix(y)+1);
        grad_x = (y - fix(y))*(interp_x2 - interp_x1) + interp_x1;

        interp_y1 = (x - fix(x))*(gy(fix(x)+1, fix(y)) - gy(fix(x), fix(y))) +
gy(fix(x), fix(y));
        interp_y2 = (x - fix(x))*(gy(fix(x)+1, fix(y)+1) - gy(fix(x), fix(y)+1)) +
gy(fix(x), fix(y)+1);
        grad_y = (y - fix(y))*(interp_y2 - interp_y1) + interp_y1;

        % update path point
        path(i,1) = x - scale * grad_x;
        path(i,2) = y - scale * grad_y;
    end
    % optionally visualize every few iterations

```

```

    if mod(iter,50)==0
        fprintf('Iteration %d complete\n', iter);
    end
end

```

```

Iteration 50 complete
Iteration 100 complete
Iteration 150 complete
Iteration 200 complete

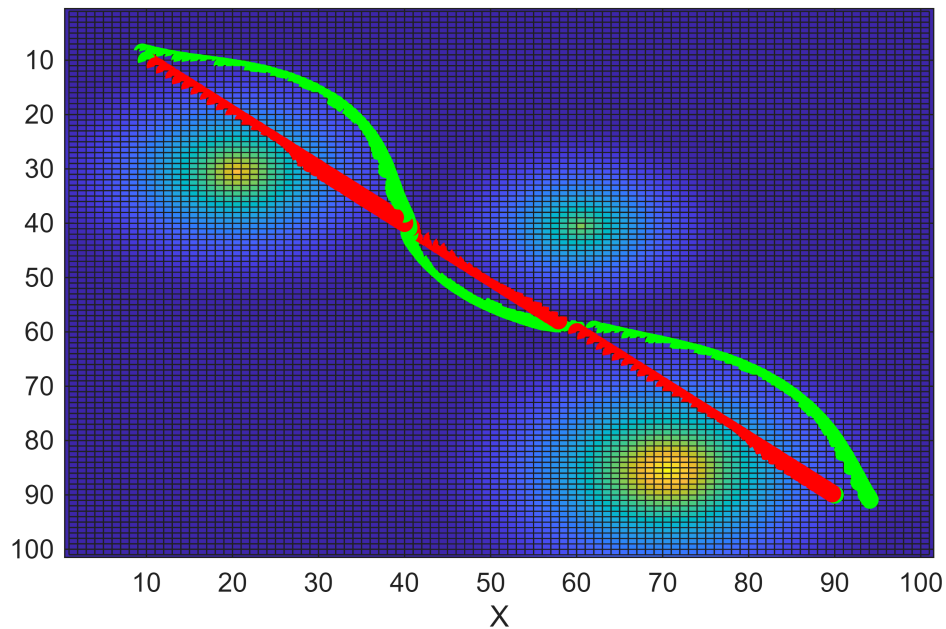
```

```

%% plot the trajectories
path_values = zeros(tt,1);
for i=1:tt
    path_values(i)=obs_cost(floor(path(i,1)),floor(path(i,2)));
end
figure(1)
hold on;
plot3(path(:,1),path(:,2),path_values, '.g', 'MarkerSize', 20);

hold off;

```



Part B

```

%% create obstacle field

clear all
waypoints=300;
N=101;
OBST = [20,30;60,40;70,85];
epsilon = [25; 20; 30];

```

```

obs_cost = double(zeros(N));
for i=1:size(OBST,1)

    t = zeros(N);
    t(OBST(i,1),OBST(i,2)) = 1; %point obstacles

    t_cost = double(bwdist(t));
    t_cost(t_cost>epsilon(i))=epsilon(i);
    t_cost = 1/(2*epsilon(i))*(t_cost-epsilon(i)).^2;

    obs_cost = obs_cost + t_cost(1:N, 1:N);
end

figure(1)
imagesc(obs_cost')
% obstacle cost gradient
gx = diff(double(obs_cost),1,1);
gy = diff(double(obs_cost),1,2);
hold on

figure(1);
surface(1:N,1:N,double(obs_cost'));
xlabel('X')
hold on;

%% initial path
%world params
SX = 10; % START
SY = 10;
GX = 90; % GOAL
GY = 90;

traj = zeros(2,waypoints);
traj(1,1)=SX;
traj(2,1)=SY;
dist_x = GX-SX;
dist_y = GY-SY;
for i=2:waypoints
    traj(1,i)=traj(1,i-1)+dist_x/(waypoints-1);
    traj(2,i)=traj(2,i-1)+dist_y/(waypoints-1);
end

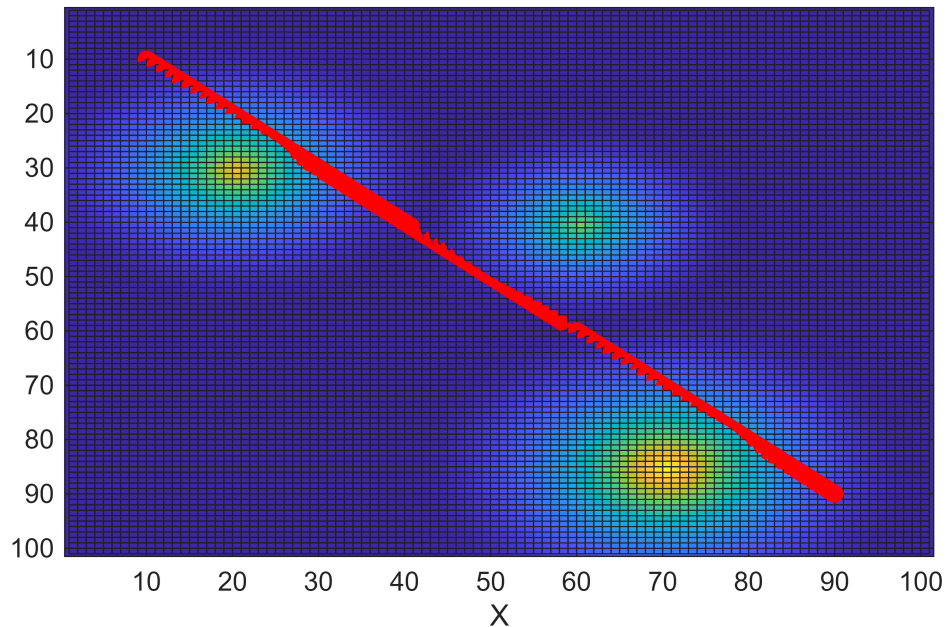
path_init = traj';
tt=size(path_init,1);
path_init_values = zeros(size(path_init,1),1);
for i=1:tt
    path_init_values(i)=obs_cost(floor(path_init(i,1)),floor(path_init(i,2)));

```

```

end
plot3(path_init(:,1),path_init(:,2),path_init_values, '.r', 'MarkerSize',20);
hold on

```



```

path = path_init;

%% Optimize it...
% your code comes here

path_b = path_init;

w_obs = 0.8;    % weight for obstacle cost gradient
w_smo = 4.0;    % weight for smoothness (xi - xi-1)
alpha = 0.1;    % step-size scaling
max_iter = 500;

for iter = 1:max_iter

    for i = 2:tt-1
        x = path_b(i,1); y = path_b(i,2);

        if x < 1 || x > N-1 || y < 1 || y > N-1
            continue;
        end

        ix = fix(x); iy = fix(y);
        interp_x1 = (x - ix) * (gx(ix+1, iy) - gx(ix, iy)) + gx(ix, iy);
        interp_x2 = (x - ix) * (gx(ix+1, iy+1) - gx(ix, iy+1)) + gx(ix, iy+1);
        grad_x = (y - iy) * (interp_x2 - interp_x1) + interp_x1;
    end
end

```



```

    interp_y1 = (x - ix) * (gy(ix+1, iy) - gy(ix, iy)) + gy(ix, iy);
    interp_y2 = (x - ix) * (gy(ix+1, iy+1) - gy(ix, iy+1)) + gy(ix, iy+1);
    grad_y = (y - iy) * (interp_y2 - interp_y1) + interp_y1;

    grad_obs = [grad_x, grad_y];

    grad_smooth = (path_b(i,:) - path_b(i-1,:));

    total_step = - w_obs * grad_obs - w_smo * grad_smooth;

    path_b(i,:) = path_b(i,:) + alpha * total_step;
end

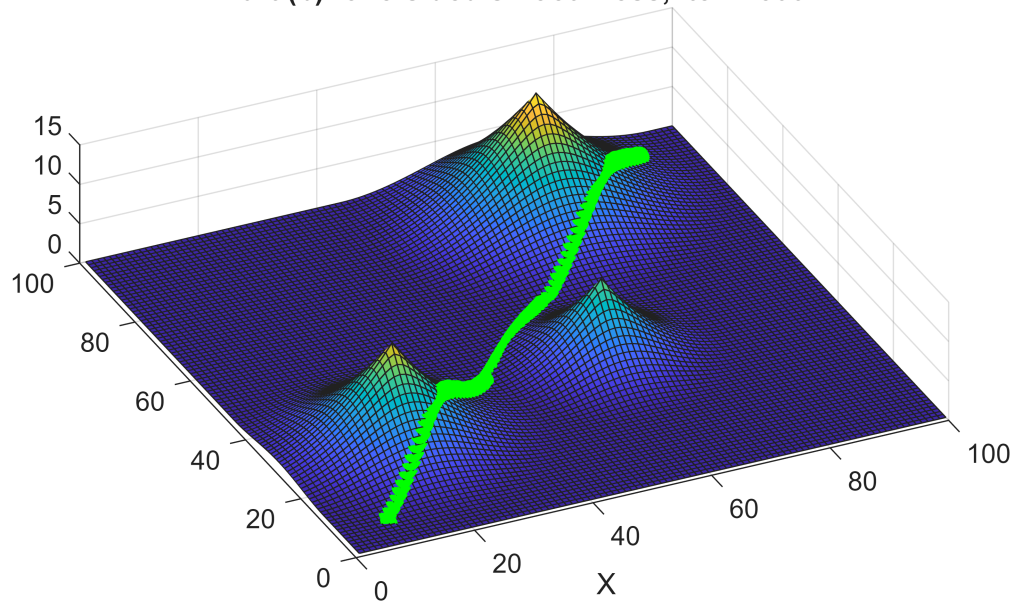
% quick snapshots
if iter == 100 || iter == 200 || iter == 500
    pv = zeros(tt,1);
    for k = 1:tt
        pv(k) = obs_cost(floor(path_b(k,1)), floor(path_b(k,2)));
    end
    figure(10); if iter==100, clf; end
    surf(1:N,1:N,double(obs_cost')), view(-25,70); hold on;
    xlabel('X'); xlim([0 100]); ylim([0 100]);
    plot3(path_init(:,1), path_init(:,2), path_init_values, '.r', 'MarkerSize',
12);
    plot3(path_b(:,1), path_b(:,2), pv, '.g', 'MarkerSize',
18);
    title(sprintf('Part (b): one-sided smoothness, iter = %d', iter));
    hold off;
    drawnow;
end
end

%% plot the trajectories
path_values = zeros(tt,1);
for i=1:tt
    path_values(i)=obs_cost(floor(path(i,1)),floor(path(i,2)));
end
figure(1)
hold on;
plot3(path(:,1),path(:,2),path_values,'.g','MarkerSize',20);

hold off;

```

Part (b): one-sided smoothness, iter = 500



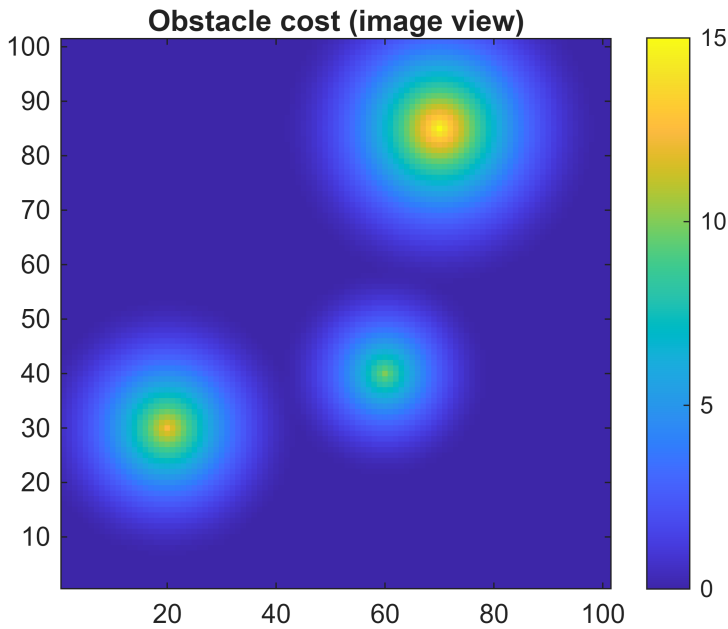
Part C

```
% create obstacle field
clear all;

waypoints = 300;
N = 101;
OBST = [20,30; 60,40; 70,85];
epsilon = [25; 20; 30];

obs_cost = double(zeros(N));
for i = 1:size(OBST,1)
    t = zeros(N);
    t(OBST(i,1), OBST(i,2)) = 1;           % point obstacle
    t_cost = double(bwdist(t));
    t_cost(t_cost > epsilon(i)) = epsilon(i);
    t_cost = 1/(2*epsilon(i)) * (t_cost - epsilon(i)).^2;
    obs_cost = obs_cost + t_cost(1:N, 1:N);
end

figure(1); clf;
imagesc(obs_cost'); axis image xy; colormap parula; colorbar;
title('Obstacle cost (image view)'); hold on;
```



```
% Discrete gradients (two flavors)
gx = diff(double(obs_cost),1,1); % (N-1) x N
gy = diff(double(obs_cost),1,2); % N x (N-1)
[FX,FY] = gradient(obs_cost); % N x N (centered finite-diff)

figure(2); clf;
surface(1:N,1:N,double(obs_cost')); shading interp; view(-25,70);
xlabel('X'); ylabel('Y'); zlabel('Cost'); title('Obstacle surface'); hold on;

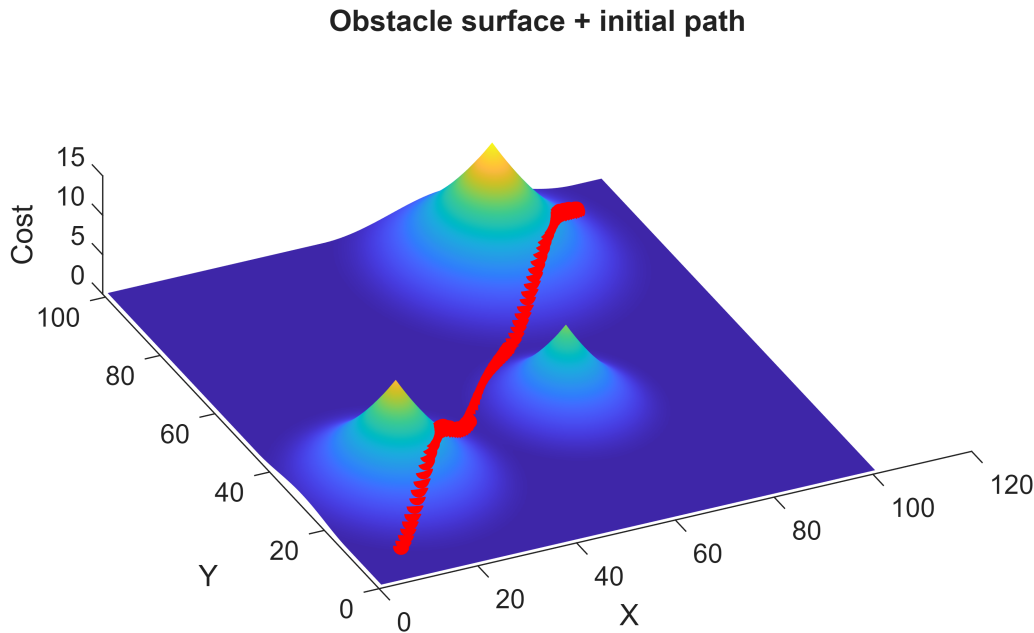
%% initial path (straight line)
SX = 10; SY = 10; % START
GX = 90; GY = 90; % GOAL

traj = zeros(2,waypoints);
traj(1,1) = SX; traj(2,1) = SY;
dist_x = GX - SX; dist_y = GY - SY;
for i = 2:waypoints
    traj(1,i) = traj(1,i-1) + dist_x/(waypoints-1);
    traj(2,i) = traj(2,i-1) + dist_y/(waypoints-1);
end
path_init = traj.'; % (tt x 2)
tt = size(path_init,1);

path_init_values = zeros(tt,1);
for i = 1:tt
    % clamp to grid just in case
    xi = min(max(floor(path_init(i,1)),1),N);
    yi = min(max(floor(path_init(i,2)),1),N);
    path_init_values(i) = obs_cost(xi, yi);
```

end

```
plot3(path_init(:,1), path_init(:,2), path_init_values, '.r', 'MarkerSize', 18);  
title('Obstacle surface + initial path');
```



```
%% ----- Q6(c) weights & step -----  
w_obs = 0.8;           % obstacle gradient weight  
w_smo = 4.0;           % smoothness weight  
alpha = 0.1;           % step size  
max_iter = 5000;  
snap_iters = [100, 5000]; % required snapshots  
  
%% ===== Method 1: Floor + Gradient =====  
% Uses integer (floor) indexing into FX, FY; two-sided smoothness:  
% grad_smooth(i) = -x_{i-1} + 2 x_i - x_{i+1} (per assignment)  
path1 = path_init;  
  
figure(3); clf;  
surface(1:N,1:N,double(obs_cost')); shading interp; view(-25,70);  
xlabel('X'); ylabel('Y'); zlabel('Cost');  
title('Method 1: Floor + Gradient'); xlim([0 100]); ylim([0 100]); hold on;  
plot3(path_init(:,1), path_init(:,2), path_init_values, '.r', 'MarkerSize', 18);  
  
for k = 1:max_iter  
    pre_path = path1; % keep previous path for smoothness term  
    for i = 2:tt-1  
        % clamp indices to valid grid (1..N)  
        xi = min(max(floor(path1(i,1)),1),N);  
        yi = min(max(floor(path1(i,2)),1),N);
```

```

% obstacle gradient (uphill) from FX,FY at integer cell
grad_obs = [FX(xi, yi), FY(xi, yi)];

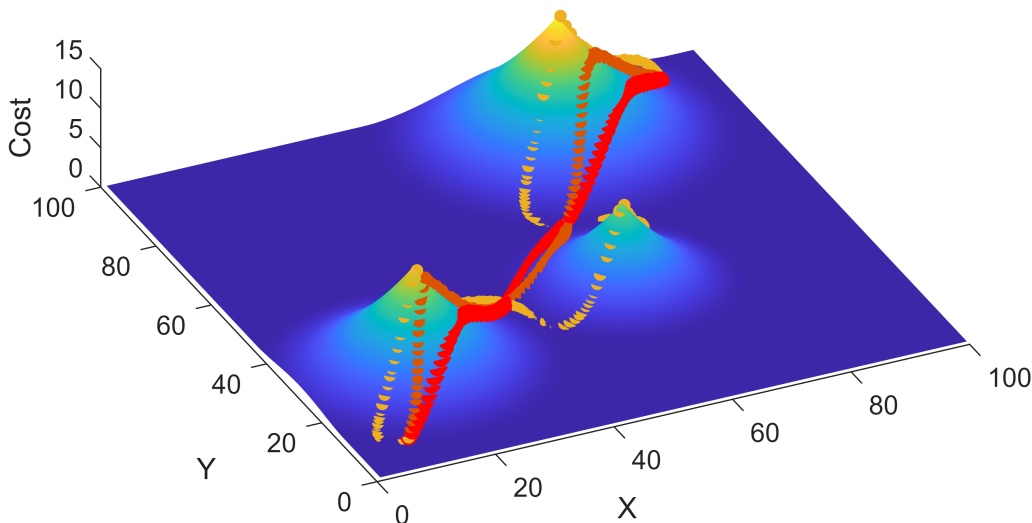
% two-sided smoothness
grad_smooth = -pre_path(i-1,:) + 2*pre_path(i,:) - pre_path(i+1,:);

% total step: negative of weighted gradients
total_step = - w_obs*grad_obs - w_smo*grad_smooth;
path1(i,:) = path1(i,:) + alpha * total_step;
end

if ismember(k, snap_iters)
    pv = zeros(tt,1);
    for j = 1:tt
        xi = min(max(floor(path1(j,1)),1),N);
        yi = min(max(floor(path1(j,2)),1),N);
        pv(j) = obs_cost(xi, yi);
    end
    plot3(path1(:,1), path1(:,2), pv, '.', 'MarkerSize',14);
    drawnow;
end
end
end

```

Method 1: Floor + Gradient



```

%% ===== Method 2: Bilinear Interpolation + pre_path =====
% Interpolates gx, gy (the forward diffs) bilinearly; uses same two-sided
smoothness.
path2 = path_init;

figure(4); clf;
surface(1:N,1:N,double(obs_cost')); shading interp; view(-25,70);

```

```

xlabel('X'); ylabel('Y'); zlabel('Cost');
title('Method 2: Bilinear Interpolation'); xlim([0 100]); ylim([0 100]); hold on;
plot3(path_init(:,1), path_init(:,2), path_init_values, '.r', 'MarkerSize', 18);

for k = 1:max_iter
    pre_path2 = path2;

    for i = 2:tt-1
        x = path2(i,1); y = path2(i,2);

        % keep interior points inside [1, N-1] so (ix+1, iy+1) are valid
        x = min(max(x, 1), N-1);
        y = min(max(y, 1), N-1);
        ix = fix(x); iy = fix(y);

        % ---- bilinear interpolation for gx (size (N-1) x N) ----
        % along x first (between ix and ix+1), then y between iy and iy+1
        interp_x1 = (x - ix)*(gx(ix+1, iy) - gx(ix, iy)) + gx(ix, iy);
        interp_x2 = (x - ix)*(gx(ix+1, iy+1) - gx(ix, iy+1)) + gx(ix, iy+1);
        grad_x = (y - iy)*(interp_x2 - interp_x1) + interp_x1;

        % ---- bilinear interpolation for gy (size N x (N-1)) ----
        interp_y1 = (x - ix)*(gy(ix+1, iy) - gy(ix, iy)) + gy(ix, iy);
        interp_y2 = (x - ix)*(gy(ix+1, iy+1) - gy(ix, iy+1)) + gy(ix, iy+1);
        grad_y = (y - iy)*(interp_y2 - interp_y1) + interp_y1;

        grad_obs = [grad_x, grad_y]; % uphill
        grad_smooth = -pre_path2(i-1,:) + 2*pre_path2(i,:) - pre_path2(i+1,:);

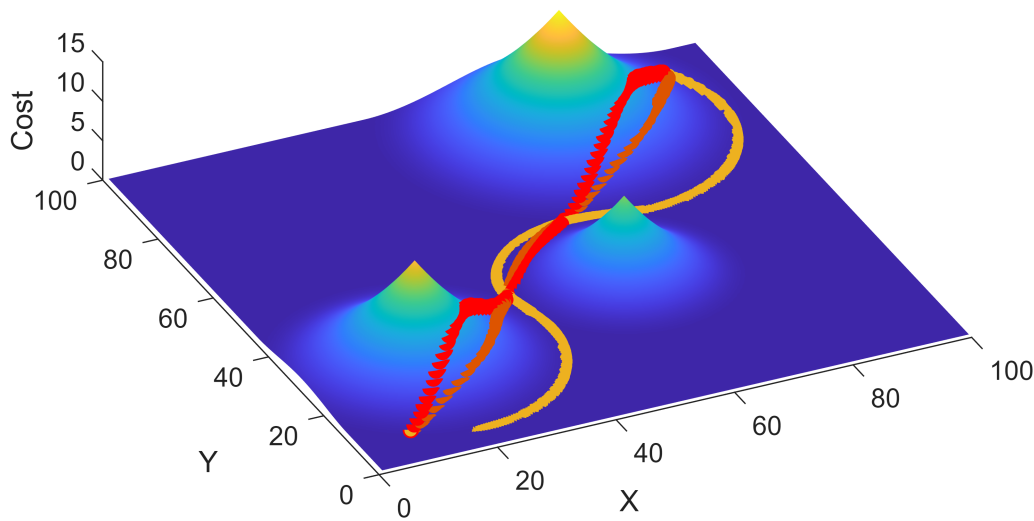
        total_step = -w_obs*grad_obs - w_smo*grad_smooth;
        path2(i,:) = path2(i,:) + alpha * total_step;

        % (optional) keep path inside the grid for display
        path2(i,1) = min(max(path2(i,1), 1), N-1);
        path2(i,2) = min(max(path2(i,2), 1), N-1);
    end

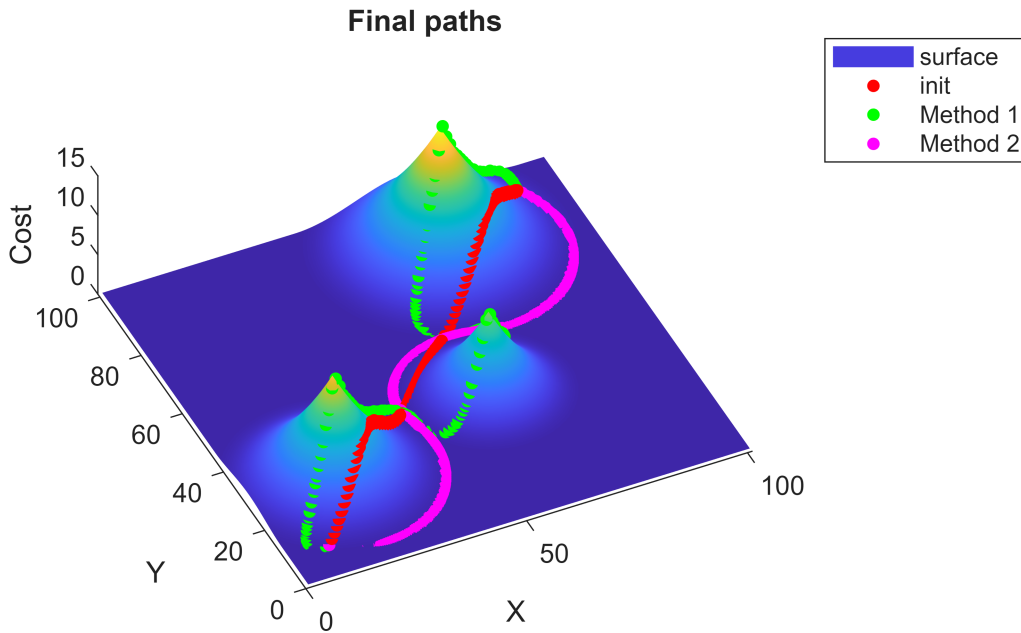
    if ismember(k, snap_iters)
        pv = zeros(tt,1);
        for j = 1:tt
            xi = min(max(floor(path2(j,1)), 1), N);
            yi = min(max(floor(path2(j,2)), 1), N);
            pv(j) = obs_cost(xi, yi);
        end
        plot3(path2(:,1), path2(:,2), pv, '.', 'MarkerSize', 14);
        drawnow;
    end
end
end

```

Method 2: Bilinear Interpolation



```
%% (Optional) overlay final 3D trajectories on surface
figure(5); clf;
surface(1:N,1:N,double(obs_cost')); shading interp; view(-25,70);
xlabel('X'); ylabel('Y'); zlabel('Cost'); hold on; title('Final paths');
pv1 = arrayfun(@(k) obs_cost(min(max(floor(path1(k,1)),1),N),
min(max(floor(path1(k,2)),1),N)), 1:tt).';
pv2 = arrayfun(@(k) obs_cost(min(max(floor(path2(k,1)),1),N),
min(max(floor(path2(k,2)),1),N)), 1:tt).';
plot3(path_init(:,1), path_init(:,2), path_init_values, '.r','MarkerSize',14);
plot3(path1(:,1), path1(:,2), pv1, '.g','MarkerSize',14);
plot3(path2(:,1), path2(:,2), pv2, '.m','MarkerSize',14);
legend('surface','init','Method 1','Method 2'); hold off;
```



Part D

No—the final path need **not** be the same. With gradient-based optimization on a **non-convex** cost landscape (obstacle costs + smoothness), the algorithm converges to a **local minimum** within the basin of attraction of the chosen initial path. Different starting paths can land in different basins and yield **different locally optimal** (and comparably “good”) paths.

Part E

Increase the obstacle cost influence by enlarging each obstacle’s radius (ϵ) or increasing its weight (for example, from 0.8 to 1.5), so the path is pushed farther away from high-cost regions.